



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



GUÍA PRÁCTICA SEMANA 04

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad I: Desarrollo Web Frontend

Tema: JavaScript C TypeScript, Manipulación del DOM, Funciones Avanzadas (Closures, IIFE, Arrow), Canvas API y Animaciones

Modalidad: Presencial / Laboratorio

Fecha: 30.04.2026

Duración: 90 minutos

APELLIDOS Y NOMBRES: Cruz Cruz Alexander Jhon

OBJETIVO DE LA PRÁCTICA

Manipular el Árbol DOM dinámicamente con JavaScript moderno (ES6+), aplicar estructuras de control, funciones flecha, autoinvocadas (IIFE) y closures para gestión de estado, e implementar una animación interactiva usando Canvas API. Comprender el motor de ejecución JS, el Event Loop, el ciclo de renderizado y estrategias de optimización (requestAnimationFrame, prevención de fugas de memoria).

RECURSOS REQUERIDOS

- Visual Studio Code (Extensiones: Live Server, JavaScript/TypeScript Language Features, ESLint, Canvas Preview)
- Navegador Chrome/Firefox + DevTools (Consola, Elements, Performance, Memory)
- TypeScript (opcional: tsc global o playground online para validación de tipos)
- Documentación oficial: MDN Web Docs (DOM, Canvas, requestAnimationFrame)
- Conexión a internet

PARTE 1: EJERCICIO PASO A PASO

Paso	Actividad	Tiempo	Concepto Clave	Restricción/Nota Técnica
1	Crear index.html, app.js (o app.ts), estructurar UI con controles y <canvas>	5 min	Integración JS/TS, carga diferida (defer), estructura DOM	No usar frameworks ni librerías externas.



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



				Vanilla JS/TS únicamente
2	Implementar IIFE para aislamiento de scope, closures para estado de animación y arrow functions para handlers	10 min	Execution context, closure scope, this lexical binding	Documentar en comentarios cómo el closure retiene el estado entre frames
3	Manipulación DOM: querySelector, addEventListener, modificación dinámica de estilos/clases, validación de inputs	10 min	DOM API, Event Bubbling/Capturing, reflow vs repaint	Evitar style inline. Usar classList.toggle() y variables CSS para rendimiento
4	Canvas: contexto 2D, requestAnimationFrame, loop de renderizado, dibujo básico (arc, fillRect, stroke)	10 min	Event Loop, render pipeline, FPS, sincronización con display refresh	Prohibido setInterval para animación. Usar delta time (dt) para movimiento uniforme
5	Depuración y optimización: Console Profiler, Memory Snapshot, detección de listeners huérfanos, FPS counter	10 min	Garbage collection, detached nodes, cancelAnimationFrame, performance metrics	Registrar métricas en README.md. IA solo para interpretar gráficos de Performance

USO DE HERRAMIENTAS DE IA (Recomendado para optimizar el ejercicio)

Herramienta	Uso Permitido	Prompt Ejemplo	Uso Prohibido
ChatGPT/Claude/Copilot	Explicar closure vs global scope, depurar context null, optimizar	"¿Por qué mi closure pierde	"Genera el código completo



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



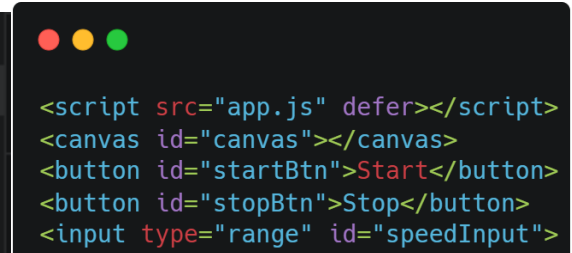
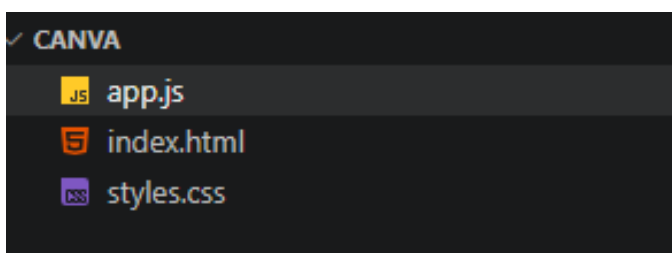
	requestAnimationFrame	referencia actxtras 100 frames? Dame patrón de encapsulació n seguro"	de una animación de partículas con canvas y controles DOM"
TypeScript Playground	Validar tipos, inferencia de funciones, interfaces para state	(Oficial)	Usar any o as any para silenciar errores sin justificación técnica
DevTools Performance	Medir FPS, detectar layout thrashing, analizar heap	(Automático)	Ignorar advertencia s de Detached DOM nodes o Long Tasks

Regla de laboratorio: El 70% de la lógica de animación y manipulación DOM debe escribirse manualmente. Cada uso de IA debe llevar comentario

Evidencias del Ejercicio Desarrollado:

- Añada capturas de pantalla de cada paso realizado en Visual Studio Code.
- Añada capturas de la Página Web implementada de manera local
- Proporcione el enlace de Github de su proyecto

Paso 01: Crear index.html, app.js (o app.ts), estructurar UI con controles y <canvas>





UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



En este paso se construye la base de la aplicación definiendo el archivo HTML con los elementos principales como el <canvas> y los controles de interacción (botones e input). Se enlaza el archivo JavaScript usando defer para asegurar que el DOM esté completamente cargado antes de ejecutar el script.

Paso 02: Implementar IIFE para aislamiento de scope, closures para estado de animación y arrow functions para handlers

```
((() => {  
  
  const state = (() => {  
    let x = 50;  
    let speed = 2;  
    let running = false;  
    let lastTime = 0;  
  
    return {  
      start: () => running = true,  
      stop: () => running = false,  
      isRunning: () => running,  
      setSpeed: (s) => speed = s,  
      update: (dt) => {  
        x += speed * dt * 0.1;  
      },  
      getX: () => x,  
      setTime: (t) => lastTime = t,  
      getTime: () => lastTime  
    };  
  })();  
  
})();
```

Se implementa una IIFE para aislar el scope y evitar contaminación global. Dentro de esta, se define un closure que encapsula el estado de la animación (posición, velocidad, tiempo y ejecución). Este closure permite que los datos persistan entre ejecuciones del loop sin ser variables globales. También se utilizan arrow functions para los handlers, aprovechando su this léxico.

Paso 03: Manipulación DOM: querySelector, addEventListener, modificación dinámica de estilos/clases, validación de inputs

```
const startBtn = document.querySelector("#startBtn");  
const stopBtn = document.querySelector("#stopBtn");  
const speedInput = document.querySelector("#speedInput");  
  
startBtn.addEventListener("click", () => state.start());  
stopBtn.addEventListener("click", () => state.stop());  
  
speedInput.addEventListener("input", (e) => {  
  const value = parseFloat(e.target.value);  
  state.setSpeed(value);  
});
```



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



En este paso se seleccionan los elementos del DOM mediante `querySelector` y se gestionan eventos con `addEventListener`. Se controlan acciones como iniciar/detener la animación y modificar la velocidad desde el input. También se valida el input para evitar valores inválidos.

Paso 04: Canvas: contexto 2D, `requestAnimationFrame`, loop de renderizado, dibujo básico (`arc`, `fillRect`, `stroke`)

```
const ctx = canvas.getContext("2d");

const draw = () => {
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  ctx.beginPath();
  ctx.arc(state.getX(), canvas.height / 2, 15, 0, Math.PI * 2);
  ctx.fill();
};

const loop = (time) => {
  const last = state.getTime() || time;
  const dt = time - last;

  state.setTime(time);

  if (state.isRunning()) {
    state.update(dt);
    draw();
  }

  requestAnimationFrame(loop);
};
```

Se obtiene el contexto 2D del canvas y se implementa un loop de animación con `requestAnimationFrame`, que sincroniza la ejecución con el refresco del navegador. Se dibuja una figura (círculo) y se actualiza su posición usando delta time (dt) para lograr movimiento uniforme independiente del FPS. Aquí intervienen el Event Loop, el render pipeline, y la sincronización con el display.



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



Paso 05: Depuración y optimización: Console Profiler, Memory Snapshot, detección de listeners huérfanos, FPS counter

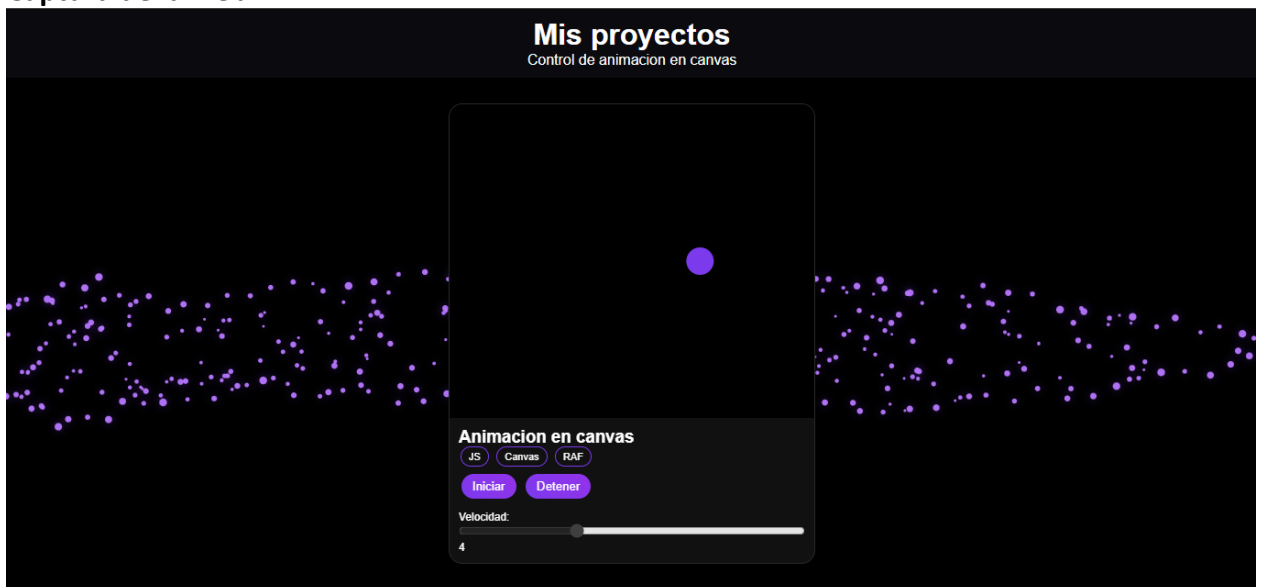


evita acumulación de frames



Finalmente, se consideran aspectos de rendimiento y memoria. Se evita la creación innecesaria de listeners, se limpia el canvas en cada frame para prevenir sobrecarga visual y se mantiene el control del loop de animación. Se puede usar `cancelAnimationFrame` para detenerlo correctamente.

Captura de la web:



Enlace del repositorio GitHub:

<https://github.com/alexandercruzjh/Canva-2D-animation>